ssh-keygen -t ed25519 -C "alice.employee@jkjabfk"
This generates the public key and private key
Ed25519 is a modern, highly secure, and very fast **public-key digital signature algorithm**.
Authentication: When you connect to a server using your Ed25519 key, the server uses your public key to verify that the signature created by your private key is valid. This process is how SSH logs you in.
Ed25519 is not the key itself; it's the method used to create the key pair.
The Private Key is the one that proves your identity, so it is stored **only** on your local machine and kept **highly secret**.
The Public Key is the one you share with the world so that servers can recognize you.
cd .ssh
ls
This is to ssh and get the public and private key

.ssh % cat id_ed25519.pub
This is to get the public key

cd ..
chmod 600 ~/.ssh/id_ed25519
chmod 700 ~/.ssh
Here we are changing the permissions

In the different shell launch the instance and get into its shell
sudo chown -R ubuntu:ubuntu ~/.ssh
sudo chmod 700 ~/.ssh
sudo chmod 600 ~/.ssh/authorized_keys
cd .ssh
ls
cat authorized_keys - we will see an authorized key here
vi authorized_keys - we add our public key to the authorized key
cat authorized_keys check again if its added
exit - exit from this vm

In the previous window run
ssh -i ~/.ssh/id_ed25519 ubuntu@192.168.64.10 - running a vm

These three commands are run on the **remote server or Virtual Machine (VM)** after you've launched it. Their purpose is to ensure the security of the SSH keys on the server by setting the correct **ownership** and **permissions** for the crucial files and folders.

Here is an explanation of each command in the context of the SSH setup:

1. Setting Ownership (`chown`)

`sudo chown -R ubuntu:ubuntu ~/.ssh`

- **`chown`**: Stands for **change owner**. This command is used to change the user and/or group ownership of a file or directory.
- **`-R`**: Stands for **recursive**. This flag tells the command to apply the change not just to the specified directory (`~/.ssh`) but also to all the files and folders **inside** it (like `authorized_keys`).
- **`ubuntu:ubuntu`**: This is the new owner. It specifies the **user** `ubuntu` and the **group** `ubuntu`.
- **`~/.ssh`**: This is the directory being acted upon—the hidden folder where SSH configuration and keys are stored on the server.

What it does:

It ensures that the entire `.ssh` directory and all its contents are owned by the specific user you are trying to log in as (in this case, the `ubuntu` user). **SSH is very strict**; it will often refuse to work if your home directory or the `.ssh` folder is not owned by you.

2. Setting Permissions for the Directory (`chmod 700`)

`sudo chmod 700 ~/.ssh`

- **`chmod`**: Stands for **change mode** or **change permissions**. This command modifies who can read, write, or execute (access/open) a file or directory.
- **`700`**: This is the numerical code for the permissions.
    - The first digit (`7`) is for the **Owner** (the `ubuntu` user). A `7` means Read, Write, and Execute permissions.
    - The second digit (`0`) is for the **Group**. A `0` means **no permissions** for the group.
    - The third digit (`0`) is for **Others** (everyone else). A `0` means **no permissions** for others.
- **`~/.ssh`**: The target directory.

What it does:

It makes the .ssh directory on the server **completely private**. Only the owner (ubuntu) can access it. This prevents other users on the same server from even seeing which public keys you have set up to allow access.

3. Setting Permissions for the Key File (chmod 600)

sudo chmod 600 ~/.ssh/authorized_keys

- **chmod**: Change permissions.
- **600**: The numerical code for the permissions on the file.
    - The first digit (6) is for the **Owner** (the ubuntu user). A 6 means **Read** and **Write** permissions (Read + Write = 4 + 2 = 6).
    - The second digit (0) is for the **Group**.
    - The third digit (0) is for **Others**.
- **~/.ssh/authorized_keys**: This is the file that contains the public keys that are allowed to log into the server.

What it does:

It makes the authorized_keys file readable and writable **only by the owner** (ubuntu). No other user on the server can read or modify this critical file. This is vital because if someone else could write to it, they could add *their* own public key and gain unauthorized access to the server.

In summary: These three commands enforce the strict security policy required by the SSH protocol: **The user's key files must only be accessible and owned by that specific user.**